



12 Parcours de graphes

► Parcours d'un graphe

12.1 Introduction

Une fois que nous sommes en présence d'un graphe, il va falloir le parcourir pour répondre à différentes questions :

- est-il possible de joindre un sommet A et un sommet B ?
- est-il possible, depuis un sommet, de rejoindre tous les autres sommets du graphe ?
- peut-on détecter la présence de cycle ou de circuit dans un graphe ?
- quel est le plus court chemin pour joindre deux sommets ?
- etc.

Les deux algorithmes principaux sont les suivants :

- le parcours en largeur – *Breadth-First Search* (BFS) – pour lequel on va commencer par visiter les sommets les plus proches du sommet initial (sommets de niveau 1), puis les plus proches des sommets de niveau 1 etc. ;
- le parcours en profondeur – *Depth-First Search* (DFS) – pour lequel on part d'un sommet initial jusqu'au sommet le plus loin. On remonte alors la pile pour explorer les ramifications.

Une des difficultés du parcours de graphe est d'éviter de tourner en rond. C'est pour cela qu'on mémorisera l'information d'avoir visité ou non un sommet. On parle aussi de marquage.

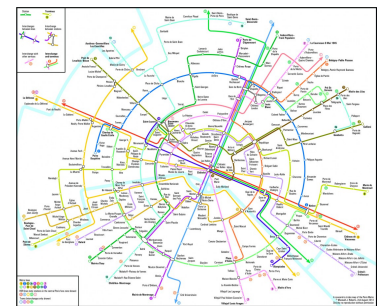


FIGURE 12.1 – Représentation circulaire du métro parisien

12.2 Parcours en largeur

12.2.1 Un premier algorithme

On propose ci-dessous un algorithme de parcours en largeur en utilisant un graphe implémenté sous forme de liste d'adjacence ainsi qu'un sommet s de départ.

```

1 def bfs(G:dict, s:str) -> None:
2     """
3     G : graphe sous forme de dictionnaire d'adjacence
4     s : sommet du graphe (Chaîne de caractère du type "S1").
5     """
6     visited = {}
7     for sommet,voisins in G.items():
8         visited[sommet] = False

```

```

9      # Le premier sommet à visiter entre dans la file
10     file = deque([s])
11     while len(file) > 0:
12         # On visite la tête de file
13         tete = file.pop()
14         # On vérifie qu'elle n'a pas été visitée
15         if not visited[tete]:
16             # Si on l'avait pas visité, maintenant c'est le cas :)
17             visited[tete] = True
18             # On met les voisins de tete dans la file
19             for v in G[tete]:
20                 file.appendleft(v)

```

Dans cet algorithme :

- ▶ on commence par créer une liste ayant pour taille le nombre de sommets. Cette liste va permettre de savoir si un sommet a été visité ou non;
- ▶ dans la file, on va commencer par ajouter le sommet initial;
- ▶ on commence alors à traiter la file en extrayant l'indice du sommet initial;
- ▶ si ce sommet n'a pas été visité, il devient visité;
- ▶ on ajoute alors dans la file l'ensemble des voisins du sommet initial;
- ▶ on continue alors de traiter la file.

Remarque

En l'état, à quoi sert cet algorithme ?

12.2.2 Applications

Exemple –

Comment connaître la distance d'un sommet s aux autres ?

```

1  def distances(G, s):
2      dist = [-1]*len(G)
3      q = deque([(s, 0)])
4      while len(q) > 0:
5          u, d = q.pop()
6          if dist[u] == -1:
7              dist[u] = d
8              for v in G[u]:
9                  q.appendleft((v, d + 1))
10     return dist

```

Exemple –

Comment connaître un plus court chemin d'un sommet s à un autre ?

```

1  def bfs(G, s):
2      pred = [-1]*len(G)
3      q = deque([(s, s)])
4      while len(q) > 0:
5          u, p = q.pop()
6          if pred[u] == -1:
7              pred[u] = p
8              for v in G[u]:
9                  q.appendleft((v, u))

```

```

10     return pred
11
12 def path(pred, s, v):
13     L = []
14     while v != s:
15         L.append(v)
16         v = pred[v]
17     L.append(s)
18     return L[::-1] # inverse le chemin

```

12.2.3 Autres formulations

```

1 def parcoursLargeur(G):
2     parcouru = {} # dictionnaire de booléens pour le marquage
3     for s in G: # Itération sur les clés de G (dictionnaire de listes)
4         parcouru[s] = False
5     f = deque() # initialisation d'une deque vide
6     for r in G: # Itération sur les clés de G (dictionnaire de listes)
7         if not parcouru[r]:
8             parcouru[r] = True
9             f.append(r)
10        while f: # Vrai tant que f est non-vide
11            s = f.popleft()
12            for ss in G[s]: # G[s] liste des successeurs de s dans G
13                if not parcouru[ss]:
14                    parcouru[ss] = True
15                    f.append(ss)

```

```

1 def parcoursLargeur(G):
2     def explorationLargeur(r):
3         f = deque() # initialisation d'une deque vide
4         f.append(r)
5         parcouru[r] = True
6         while f: # Vrai tant que f est non-vide
7             s = f.popleft()
8             for ss in G[s]: # G[s] liste des successeurs de s dans G
9                 if not parcouru[ss]:
10                    f.append(ss)
11                    parcouru[ss] = True
12     parcouru = {} # dictionnaire de booléens pour le marquage
13     for s in G.keys(): # Itération sur les clés de G (dictionnaire de listes)
14         parcouru[s] = False # initialisation du marquage
15     for r in G.keys(): # Itération sur les clés de G (dictionnaire de listes)
16         if not parcouru[r]:
17             explorationLargeur(r)

```

12.3 Parcours en profondeur

12.3.1 Un premier algorithme

On propose ci-dessous un algorithme de parcours en profondeur en utilisant un graphe implémenté sous forme de liste d'adjacence ainsi qu'un sommet s de départ.

```

1 def dfs(G, s): #
2     visited = [False]*len(G)

```

```

3 pile = [s]
4 while len(pile) > 0:
5     u = pile.pop()
6     if not visited[u]:
7         visited[u] = True
8         for v in G[u]:
9             pile.append(v)

```

Dans cet algorithme :

- ▶ on commence par créer une liste ayant pour taille le nombre de sommets. Cette liste va permettre de savoir si un sommet a été visité ou non;
- ▶ dans la pile, on va commencer par ajouter le sommet initial;
- ▶ on commence alors à traiter le sommet initial après l'avoir extrait de la pile;
- ▶ si ce sommet n'a pas été visité, il devient visité;
- ▶ on ajoute alors dans la pile l'ensemble des voisins du sommet initial;
- ▶ on continue alors de traiter la pile.

À la différence du parcours en largeur, lorsqu'on va traiter la pile, on va s'éloigner du sommet initial... avant d'y revenir quand toutes les voies auront été explorées.

12.3.2 Une autre formulation

La formulation précédente du parcours en profondeur a l'avantage d'être très proche de celle du parcours en largeur. Cependant, si on traçait l'arbre permettant de visualiser les sommets visités, on constate que l'algorithme crée des ramifications qui ne correspondent pas vraiment à un parcours en profondeur¹. Il s'agit alors de

1: <https://11011110.github.io/blog/2013/12/17/stack-based-graph-traversal.html>

```

1 def dfs(G, s): #
2     visited = [False]*len(G)
3     pile = [s]
4     while len(pile) > 0:
5         u = pile.pop()
6         if not visited[u]:
7             visited[u] = True
8             for v in G[u]:
9                 pile.append(v)

```

```

1 def dfs_2(G, s): # A VERIFIER :)
2     visited = [False]*len(G)
3     pile = [s]
4     while len(pile) > 0 :
5         u = pile.pop()
6         if not visited[u]:
7             visited[u] = True
8             pile.append(u)
9             for v in G[v] :
10                 pile.append(v)

```

12.3.3 Une autre formulation (récursive)

```

1 def dfs(G, s):
2     visited = [False]*len(G)
3     def aux(u):
4         if not visited[u]:
5             visited[u] = True
6             for v in G[u]:
7                 aux(v)
8     aux(s)

```

12.3.4 Applications

Exemple –

Lister les sommets dans l'ordre de leur visite.

Exemple –

Comment déterminer si un graphe non orienté est connexe ?

Exemple –

Comment déterminer si un graphe non orienté contient un cycle ?

Références

- ▶ Cours de Quentin Fortier <https://fortierq.github.io/itc1/>.
- ▶ Cours de JB Bianquis. Chapitre 5 : Parcours de graphes. Lycée du Parc. Lyon.
- ▶ Cours de T. Kovaltchouk. Graphes : parcours. Lycée polyvalent Franklin Roosevelt, Reims.
- ▶ https://perso.liris.cnrs.fr/vincent.nivoliers/lifap6/Supports/Cours/graph_traversal.html
- ▶ <http://mpechaud.fr/scripts/parcours/index.html>